

• COMPLETE ARCHITECTURE

Knowledge Base Architecture

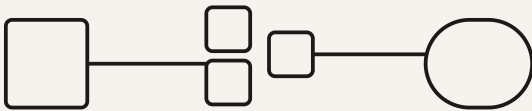
A scoped, hybrid retrieval layer on Qdrant Cloud: org, user, and agent-level Knowledge Bases sharing one collection.

• ONE-PAGE SUMMARY, FOR PRESENTING
THE WHOLE THING, IN ONE PAGE

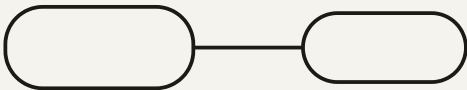
A document gets uploaded, chunked, and stored once. A question gets cleaned up, searched against three tiers at once (company, personal, agent), combined into one ranked list, permission-checked, and handed to the AI. Every stage below has a full diagram further down this page.



One database, many tenants
Every company, KB, and document lives in the same database, kept apart by tags, not separate databases.



Ingestion
Upload a document, split it into small chunks, turn each into two searchable fingerprints, and save.



Query rephrase
Clean up the question using recent chat, so "it" and "that" get resolved before searching.



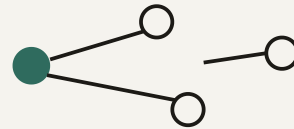
Query embedding
Turn the question into the same two fingerprints used for documents: one for meaning, one for

keywords.



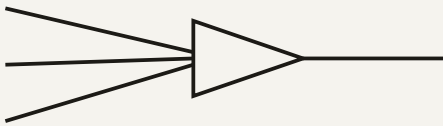
Hybrid search

Search with both fingerprints at once, scoped to just this tier's documents, in a single database call.



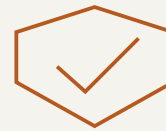
Graph expand

If turned on for that tier, also pull in chunks linked to the ones already found.



Merge + rerank

Combine the ranked results from company, personal, and agent search into one list.



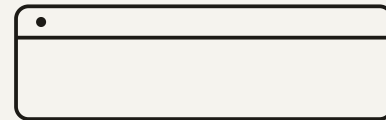
Scope filter

Check permissions one more time and drop anything this person isn't allowed to see.



Agent context

Hand the safe, ranked results to the AI as part of the conversation, and save them for later turns too.



Frontend

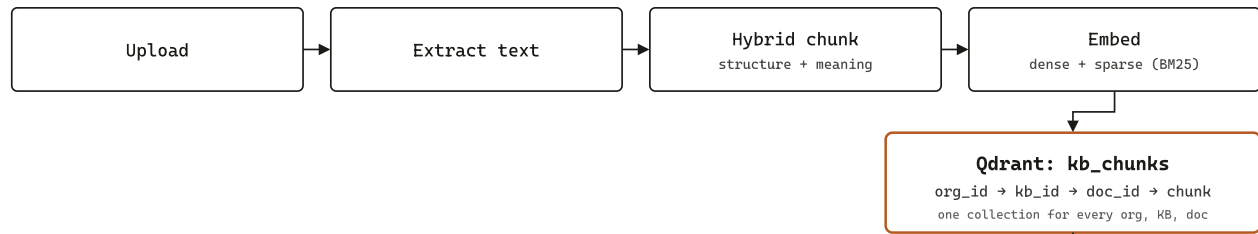
Reuses existing upload, permission, and connect-to-agent screens. New: a KB manager and showing sources in chat.

01 OVERVIEW

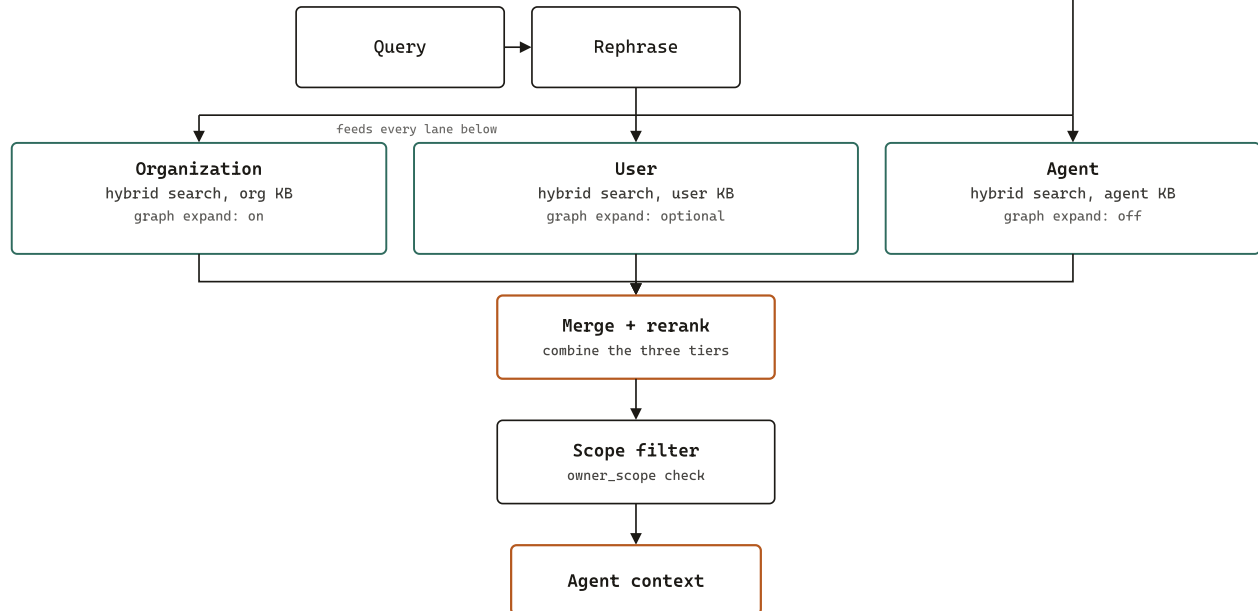
A Knowledge Base is owned at one of three tiers (organization, user, or agent), and each tier runs retrieval with its own settings. Documents land in one shared Qdrant collection, differentiated by payload rather than by separate collections. A query is rephrased, searched per tier, merged, and permission-filtered before it reaches the agent.

02 COMPLETE ARCHITECTURE

INGEST (runs once per document, regardless of tier)



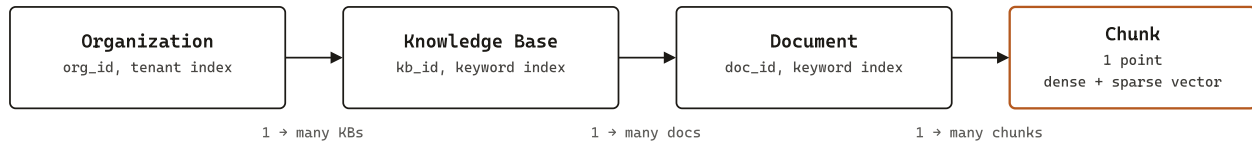
RETRIEVE (runs per query)



Ingestion (top) tags every chunk with `org_id`, `kb_id`, `doc_id` and writes it once, regardless of tier. Retrieval (bottom) runs the same rephrased query against all three tiers in parallel, each with its own hybrid-search and graph-expansion settings, then merges, reranks, and permission-filters before the agent ever sees a chunk.

03 HOW QDRANT TELLS MULTIPLE KBS AND DOCUMENTS APART

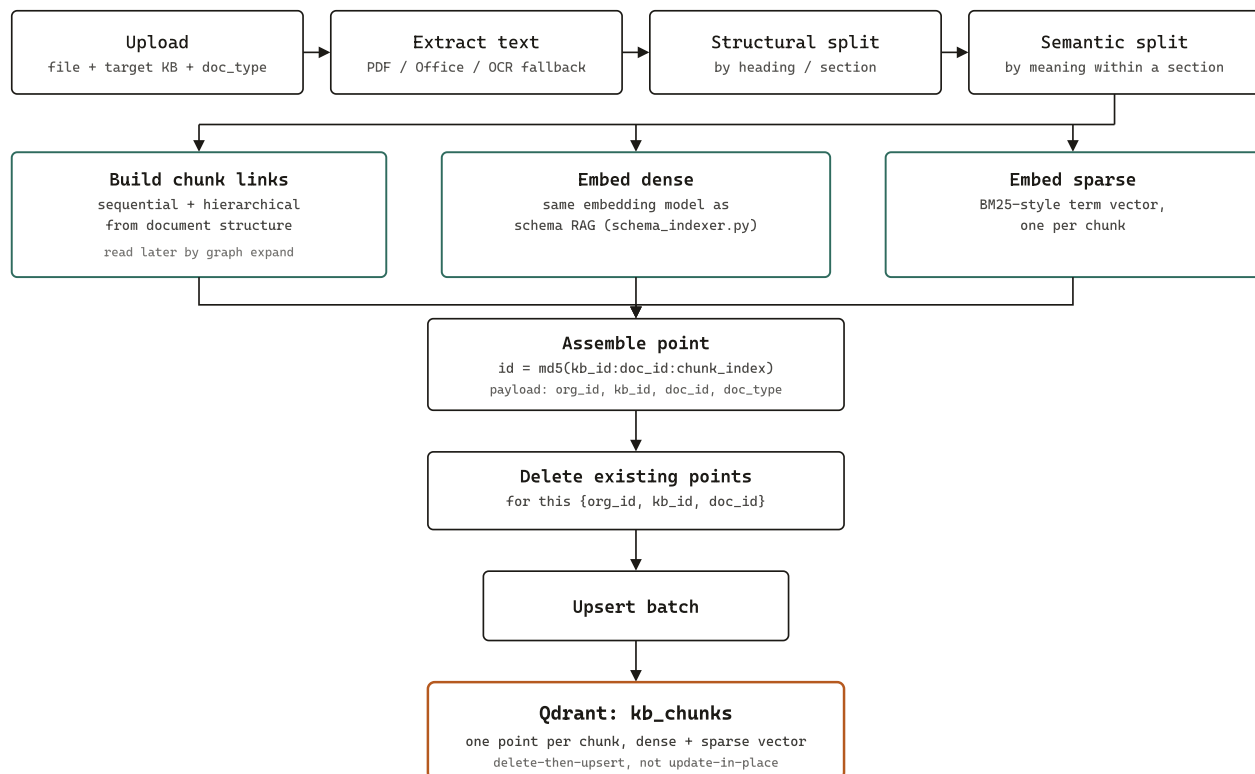
One org can have many Knowledge Bases, and one KB can have many documents. All of it lives in the single `kb_chunks` collection above, with no collection-per-org or collection-per-KB. Differentiation is entirely by payload field and index, the same mechanism already running today for DB-schema search (`ai/rag/schema_indexer.py`), one level deeper: `org_id` as the tenant index, `kb_id` as a keyword index (many per org), `doc_id` as a keyword index (many per KB).



A query for one agent filters by `org_id` (tenant), then `kb_id IN [ ... ]` for the KBs it can see, optionally `doc_id IN [ ... ]` to narrow further. No cross-collection joins, one query per tier lane.

04 INGESTION PIPELINE IN DETAIL

The four-box ingest row in the diagram above compresses ten real steps. Reused pieces are cited to the file that already does them; everything else is new.

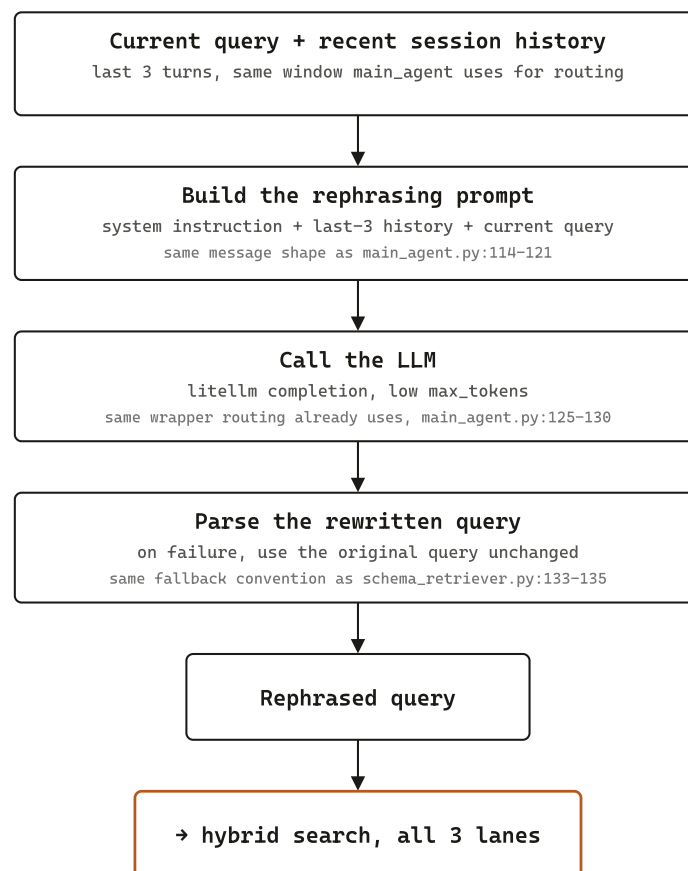


Delete-before-upsert on `{org_id, kb_id, doc_id}` mirrors `index_schema` in `ai/rag/schema_indexer.py:276-285`, so re-ingesting one document is always a clean slate rather than an accumulating duplicate. The dense embedding step reuses the same `litellm.aembedding`

call schema RAG already makes; sparse embedding and chunk-link construction are the two genuinely new pieces of work.

05 QUERY REPHRASE IN DETAIL

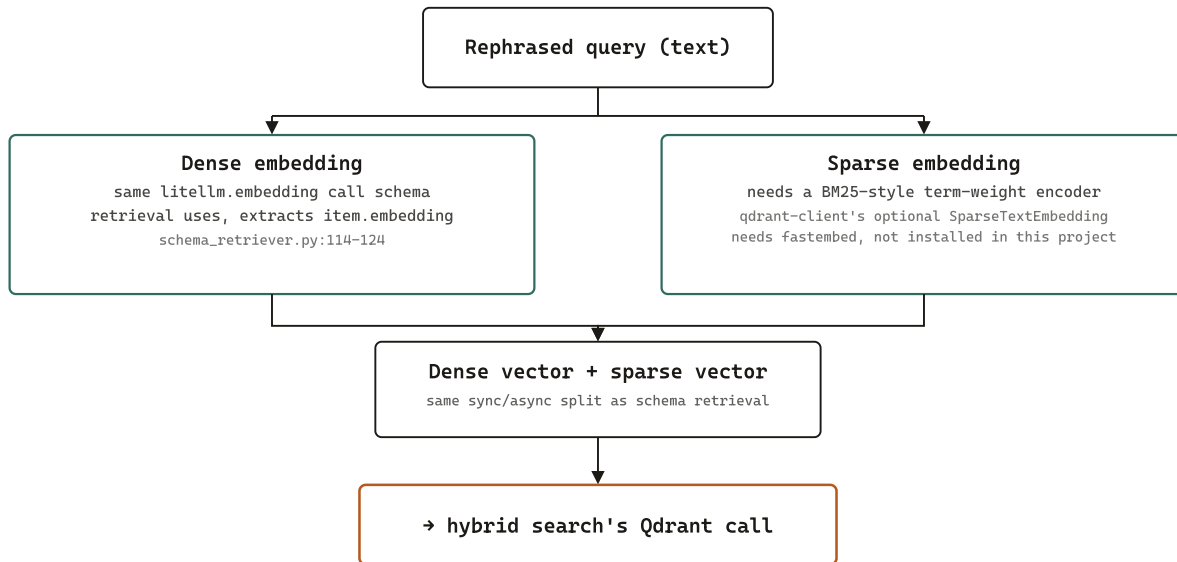
Nothing in this codebase rephrases a query today: the reuse table already flags this as net new. What exists is the shape a rephrase step would reuse, the same last-3-turns-of-history pattern `MainAgent._route` already uses for follow-up-aware routing (`ai/agents/main_agent.py:118–121`), and the same `litellm` call wrapper (`:125–130`).



This shows one rewritten query going to all three lanes, matching every other diagram in this document. Whether rephrasing should instead produce several query variants (a common RAG technique) is not decided and not shown here; adding it would mean fanning this box's output before hybrid search, not changing anything upstream of it.

06 QUERY EMBEDDING IN DETAIL

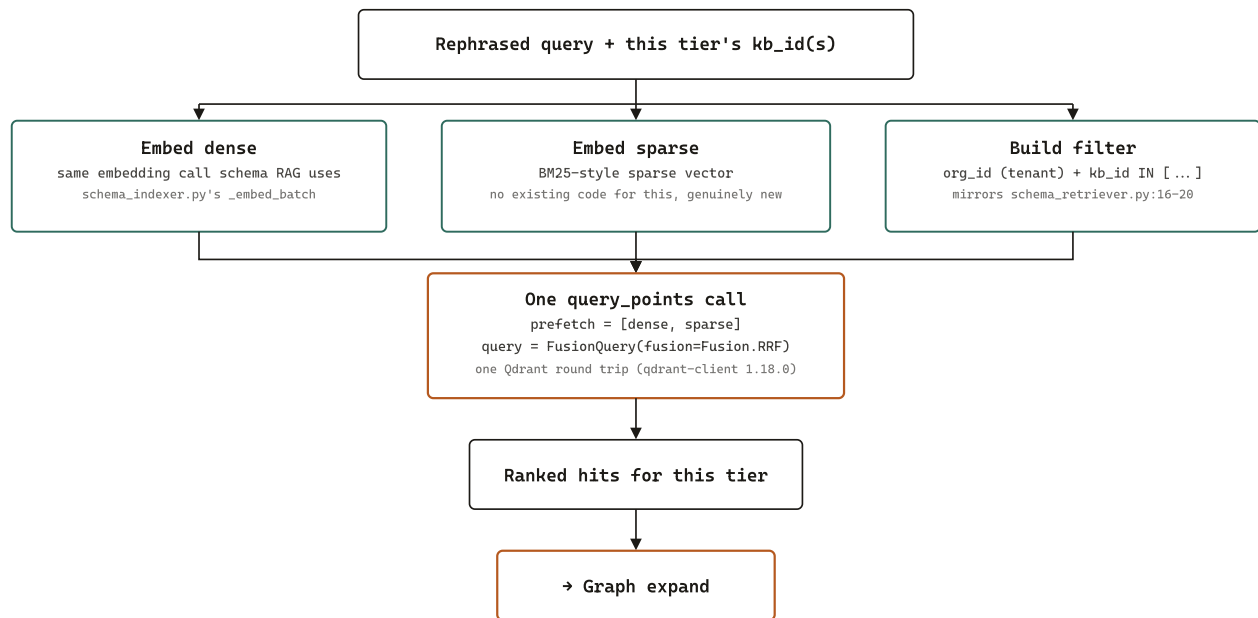
Dense query embedding already exists, in `get_relevant_schema_sync`: a plain `litellm.embedding` call on the same `settings.EMBEDDING_MODEL` used for indexing (`ai/rag/schema_retriever.py:114-124`). Sparse query embedding does not: I checked whether the installed `qdrant-client`'s optional sparse-embedding helper is available, and it is not. `qdrant_client.models.SparseTextEmbedding` resolves to `None` at runtime because it depends on the `fastembed` package, which is not a dependency of this project (absent from `pyproject.toml` and `uv.lock`).



The dense path costs nothing new: same model, same call shape, same sync-wrapped-for-async pattern the `multi_orchestration` engine already relies on because it has no event loop to await from. The sparse path is a real open decision, not a detail to fill in later: either add `fastembed` as a new dependency, or bring a BM25 encoder some other way. This document doesn't pick one.

07 HYBRID SEARCH IN DETAIL

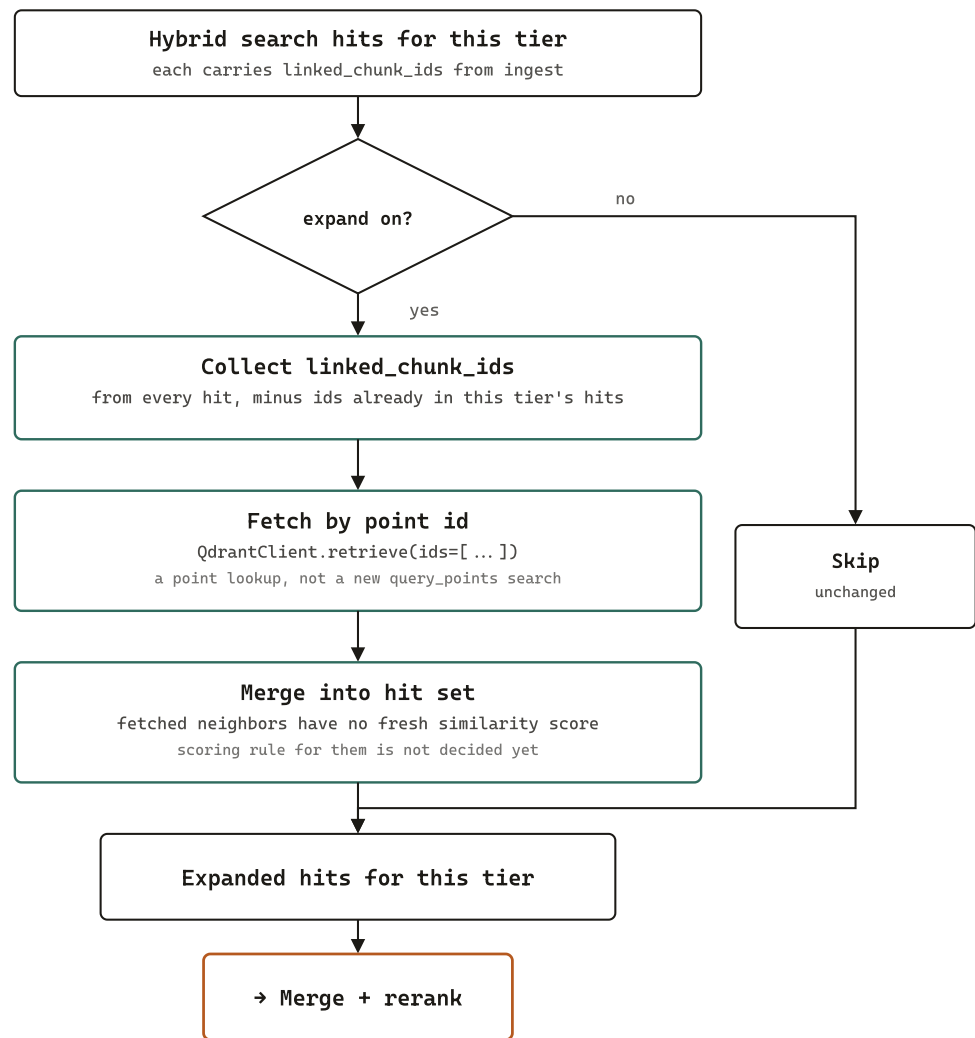
Today's search inside a tier's KB would be a single dense-only `query_points` call, the same shape `get_relevant_schema_sync` already makes (`ai/rag/schema_retriever.py`). Hybrid search adds a second, sparse vector and fuses both in one call, using capability already present in the installed `qdrant-client`, confirmed by inspecting `qdrant_client.models` directly: `Prefetch`, `FusionQuery`, `Fusion.RRF`.



The filter is the one piece already proven at scale: it's the exact tenant-plus-keyword-index pattern from schema RAG, just `kb_id` where that code has `db_conn_id`. The fusion is new wiring, not new capability: nothing in this codebase calls `Prefetch` or `FusionQuery` today, but both exist on the client already installed.

08 GRAPH EXPAND IN DETAIL

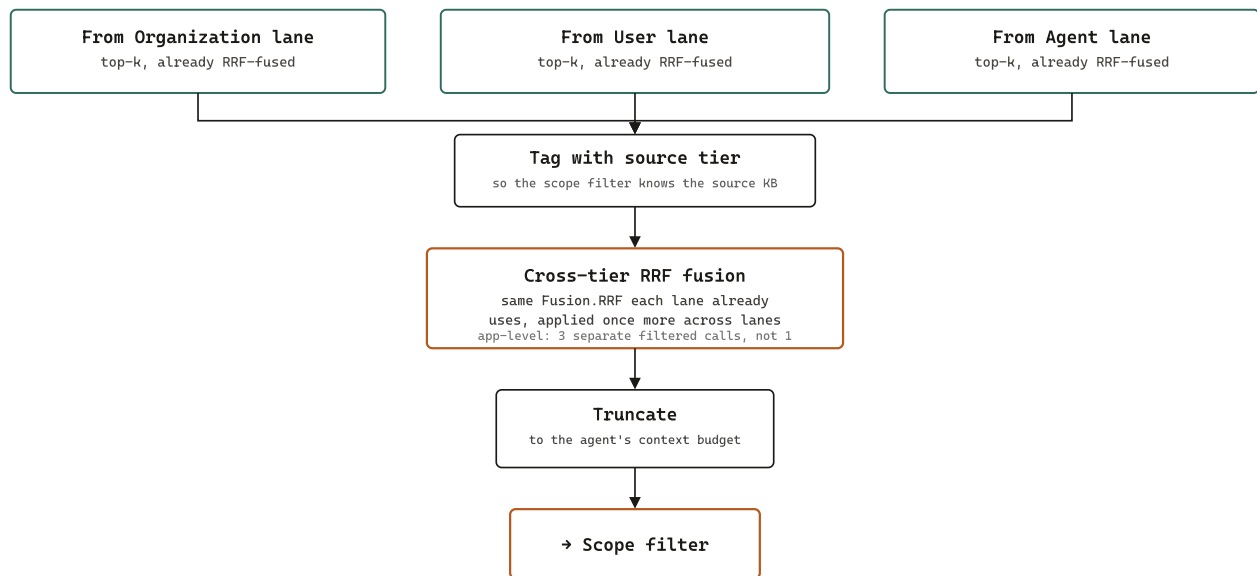
This runs inside each tier's hybrid-search lane, using the links `Build chunk links` wrote at ingest time. The pull is a direct point lookup by id (`QdrantClient.retrieve`, confirmed on the installed client), not another similarity search.



The "how many hops" question from graph link-building carries over here unresolved: this shows one hop. Scoring is unresolved too. A chunk pulled in by a link was never ranked by the search itself, and this document doesn't invent a number for what score it should carry into merge + rerank.

09 MERGE + RERANK IN DETAIL

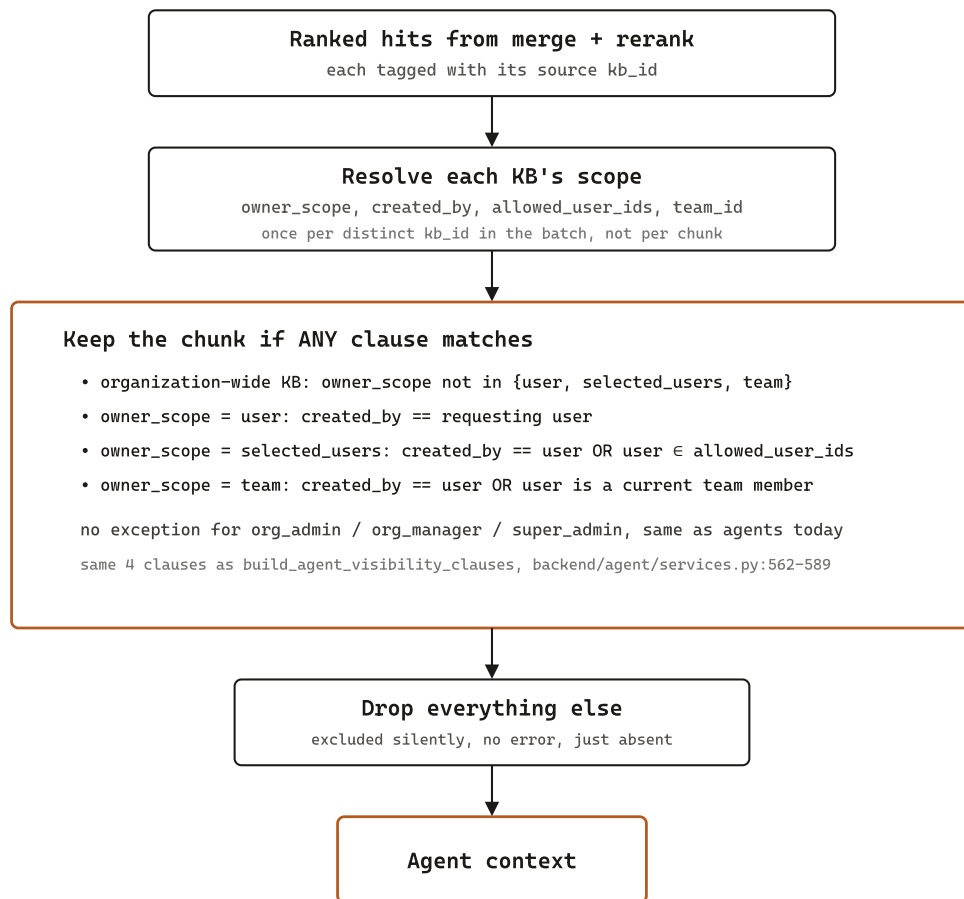
Each tier lane already returns a ranked list: its own dense and sparse results fused with Qdrant's **Fusion.RRF** inside that lane's single `query_points` call. Merge + rerank is what happens next: three independently-ranked lists, from three separate filtered Qdrant calls, need to become one.



The tag-with-tier step exists only because the scope filter runs after merge, not before: a chunk still needs to know which KB it came from to be checked against `owner_scope`. The fusion step is deliberately the same RRF technique already used inside each lane, not a separate reranker model. Nothing in this codebase calls a dedicated reranker today, so this document doesn't assume one.

10 SCOPE FILTER IN DETAIL

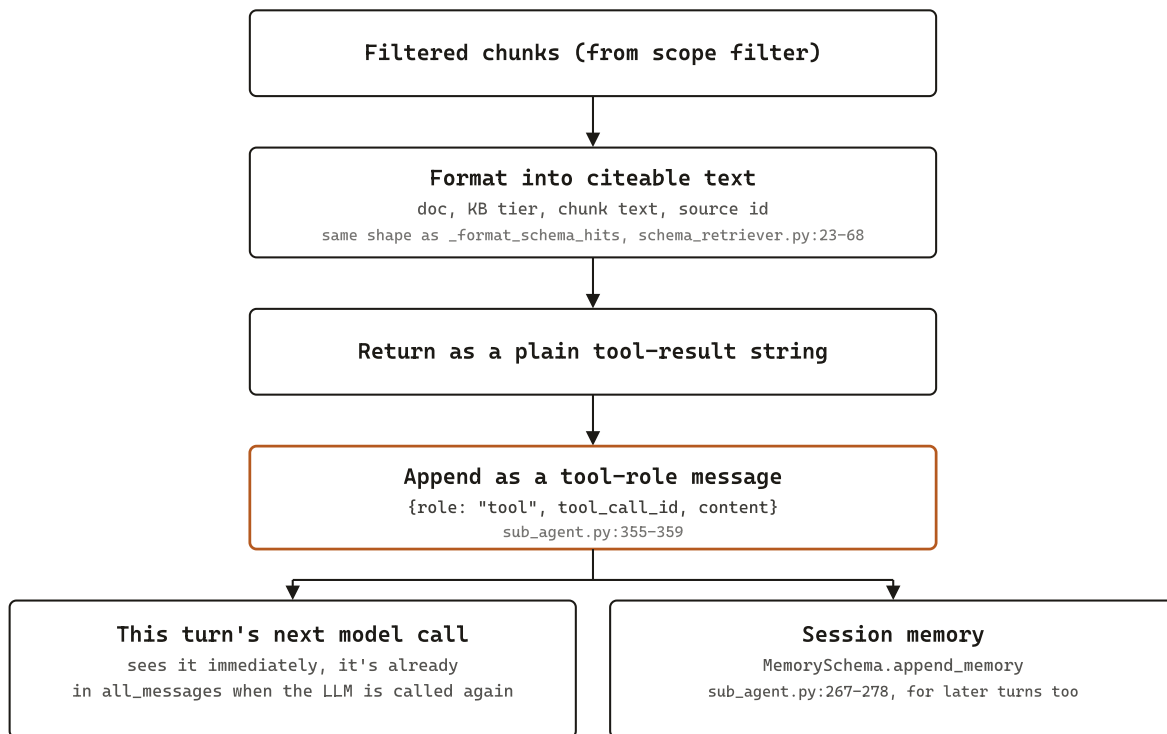
This is not a new rule: it's the exact visibility check agents already use, applied to chunks instead of agents. `build_agent_visibility_clauses` in `backend/agent/services.py:562-589` is the single visibility rule for the whole app today (reused as-is by chat, direct_agent, and widget code); a KB's `owner_scope` would be checked the same way.



The four clauses are copied, not reinvented: organization-wide KBs are visible to everyone in the org; a personal KB only to its creator; a selected-users KB to its creator or anyone named in `allowed_user_ids`; a team KB to its creator or a current member of `team_id`. Per the comment on `build_agent_visibility_clauses`, there is deliberately no admin bypass: a personal KB stays private even from `org_admin/org_manager/super_admin`.

11 AGENT CONTEXT IN DETAIL

This is also not new machinery: it's the exact path `search_schema` already uses today, read directly from `ai/agents/sub_agent.py` and `ai/rag/schema_retriever.py`. A KB retrieval tool would return a plain string the same way and travel the same route into the model.



Nothing here is a new mechanism: a KB search tool returns a string exactly like `execute_search_schema_sync` does today, and that string travels the same fork every tool result already does: into this turn's next LLM call, and into the append-only per-session memory log so it survives past this turn.

12 STAYING RELIABLE AND SCALABLE, NOT JUST CORRECT ON PAPER

- Ingestion must be a background task, not an inline request.** Chunking + dual embedding + link-building for a large document can't run inside the upload HTTP request. Reuse the exact pattern `backend/dbconnection/services.py` already uses for schema indexing: `asyncio.create_task( ... )` fires the pipeline, the request returns immediately (:453-455), and a status flag gets set on completion (`schema_indexed: True` at :397, would be `kb_document_indexed` here). This isn't shown anywhere above and needs to be.
- A failed lane shouldn't fail the query.** If one tier's Qdrant call errors or times out, merge + rerank should proceed with whichever lanes succeeded, the same discipline `get_relevant_schema_sync` already follows: catch, log, degrade, never crash the agent turn over a search failure (`schema_retriever.py:133-135`). This isn't in the pipeline diagrams above and should be, wherever hybrid search is actually implemented.
- One collection scales by tenant count, not by row count.** `org_id` as a tenant index plus a per-tenant HNSW subgraph (`payload_m`) is Qdrant's own recommended pattern for many tenants sharing one collection, already proven at production scale on `schema_tables`.

`kb_chunks` inherits that property directly; adding more orgs, KBs, or documents doesn't change the shape of the solution.

4. **Sparse search is additive, not a blocker.** Until a sparse encoder is chosen (section 06's open item), hybrid search's `Prefetch` list can run dense-only, the exact single-vector shape schema RAG already runs in production today. This ships in a dense-only phase first rather than waiting on the missing piece.
5. **Delete-then-upsert has a real, inherited gap.** Re-ingesting a document briefly leaves zero points for it between the delete and the upsert, the same behavior `schema_indexer.py` already has for DB schemas today. That's been fine at schema scale (dozens of tables); at document scale, a single doc can be thousands of chunks, so that window gets longer and is worth re-checking before assuming it carries over unchanged.

13 FRONTEND

Everything above is backend and `ai/` only. The frontend (One-AI-UI: Next.js 16 App Router, React 19, TypeScript, Tailwind v4 with shadcn) already has three of the four UI patterns this feature needs, just not wired to a Knowledge Base yet. Read directly from the actual components, not designed from scratch.

Frontend piece	Status	Where
KB creation form (owner_scope, allowed_user_ids, team_id)	mirrors existing pattern	<code>agent-builder-popup.tsx:18-22</code> (scope pills), edit page's <code>allowed_user_ids</code> checklist and <code>team_id</code> pills
Document upload (drag-drop, multipart POST)	mirrors existing pattern	<code>project-files-dialog.tsx:216-223</code> , <code>lib/api/projects.ts:76-79</code>
Connect KB to agent (browse, select, attach)	mirrors existing pattern	<code>components/mcp/agent-mcp-step.tsx</code> , the org-wide-list → expand → check → attach shape
<code>rag_ids</code> on the Agent type	not declared yet	absent from <code>AgentPublic</code> in <code>types/index.ts:99-101</code> , only <code>tool_ids/connector_ids/MCP</code> attachments exist today
<code>knowledge_base_ids</code> on the Project type	dangling stub	<code>types/index.ts:140,159,173</code> , declared, referenced nowhere else in the repo
KB management screens (list, create, delete, tier picker)	net new	no existing screen
Chunk citations in chat	net new	playground renders only markdown (<code>app/(dashboard)/client/playground/page.tsx:506-592</code>); the nearest precedent is the separate Trace link on

Frontend piece

Status

Where

`msg.trace_id` (same file, 578-587), a link out to the tracing UI, not an inline citation



The existing wizard steps, read from `app/(dashboard)/client/agents/[id]/edit/page.tsx`. Knowledge Bases (highlighted) sits naturally alongside the other resource-attachment steps, Tools, Connectors, and MCP Servers. Exact ordering is a UI decision this document doesn't make.

drafted from whiteboard photos, 2026-08-18